

Polynomial Regression Assignment

<https://github.com/Akshay-K-M/Polynomial-Regression>

Akshay K M
IMT2024014

1 Overview

The task is to predict y from the inputs in two datasets using only polynomial regression. Each training file has 1000 rows. In var1 there are six inputs and the problem says the true degree is at most 10. In var2 there are three inputs and the degree is at most 20.

The main difficulty is the number of terms. A polynomial of total degree d in n inputs has $\binom{d+n}{n}$ terms (without counting the constant). For var1 this is 461 terms at degree 5, 1715 at degree 7 and over 8000 at degree 10. For var2 it is 363 at degree 11 and 1771 at degree 20. Once the number of terms gets close to the number of rows (or goes past it), ordinary least squares stops being usable. I did not choose a degree by hand. I wrote one script that tries many degrees and several regularised models, and picks the best one using cross validation.

2 Method

2.1 Features

For a given degree d I built every monomial $x_1^{a_1} \cdots x_n^{a_n}$ with $a_1 + \cdots + a_n \leq d$ (this is the definition of degree given in the assignment). Each column was then standardised to mean 0 and standard deviation 1. This matters for the penalised models: without it, a term like x^{10} lives on a very different scale from x and the penalty treats the terms unfairly.

2.2 Models

For every degree I fitted four models:

- **Ordinary least squares (OLS).** The normal equation solution. I only used it when the number of terms was smaller than the number of training rows in a fold, since otherwise the answer is not unique.
- **Ridge regression.** The closed form $\hat{w} = (X^\top X + \alpha I)^{-1} X^\top y$ where α plays the role of $N\lambda$. I computed this with an SVD, which stays stable when there are more terms than rows. The penalty α was searched over 41 values from 10^{-4} to 10^4 .
- **Lasso (L1 penalty).** It can set coefficients exactly or nearly to zero, so it works as automatic term selection.
- **Elastic net** (L1 plus L2 penalty), with the L1 share tried at 0.1, 0.3, 0.5, 0.7 and 0.9.

For lasso and elastic net the strength λ was searched over a grid of 40 values. The grid starts at the smallest value that makes all coefficients zero and goes down by a factor of 1000.

2.3 Why closed form and not gradient descent

OLS and ridge have exact solutions, so I used them directly. Gradient descent on degree 10 or higher monomials is badly conditioned and would need a learning rate and number of epochs tuned for every degree, with no guarantee of reaching the true minimum. Hence I made this choice for high degrees. Lasso and elastic net have no closed form, so those two use coordinate descent (scikit-learn).

2.4 Choosing the degree and model

All choices were made with 5-fold cross-validation. The folds were fixed once (shuffled, seed 0) and every model and every degree was scored on the same folds, so the comparison is fair. For each model the penalty was picked by the lowest average validation MSE, and then models and degrees were compared on that same number. I also report the CV R^2 , computed as $1 - \text{MSE}/\text{Var}(y)$.

Two small rules were added to keep the search sensible:

1. The degree is increased one at a time, and the search stops when the best CV MSE has not improved for 3 degrees in a row. This is the usual U-shaped curve idea: once the validation error stops improving there is no point in going further. It also saves a lot of time, since lasso on thousands of terms is slow.
2. If several degrees are within 1% of the best CV MSE, the lowest degree is taken. The differences there are smaller than the noise in the estimate, and a simpler polynomial is the safer choice.

Finally, the winning degree, model and penalty are refitted on all 1000 training rows and used to predict the test file. No data was held out for this last step.

3 Results

3.1 var1 (six inputs)

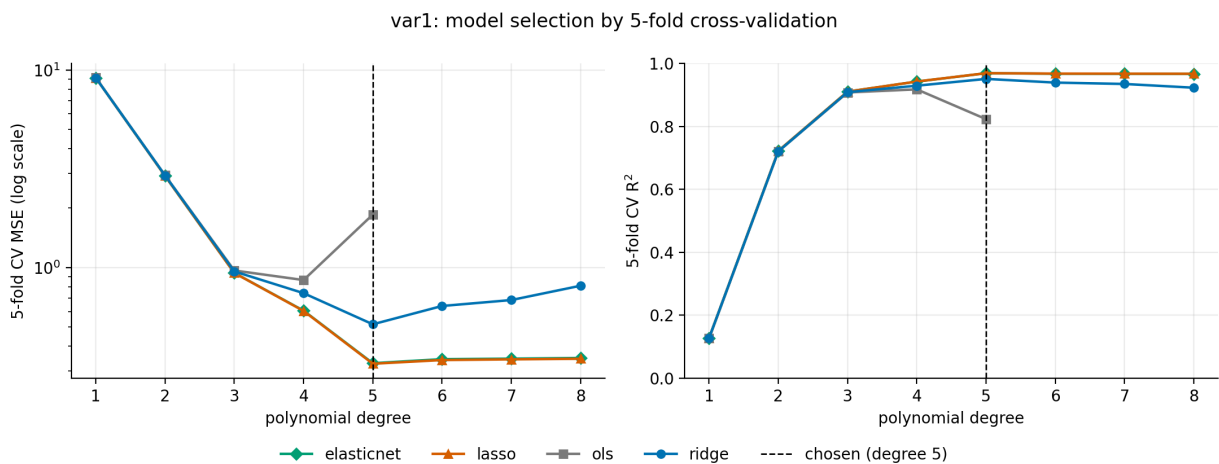


Figure 1: var1: 5-fold CV MSE (left, log scale) and CV R^2 (right) against polynomial degree for each model. The dashed line marks the chosen degree.

The CV error drops quickly up to degree 3, keeps improving until degree 5, and then goes up slightly (Figure 1). Lasso and elastic net are almost the same and clearly better than ridge. OLS

is only available up to degree 5, and at degree 5 it already does badly (CV MSE 1.85) because 461 terms is too many for the 800 rows in each training fold.

The search stopped after degree 8, since degrees 6, 7 and 8 did not beat degree 5. The chosen model is **degree 5 with lasso**. Lasso kept 125 of the 461 terms, which fits the idea that the true polynomial uses only a part of the possible terms.

Model	CV MSE	CV R^2	Penalty chosen
OLS	1.855	0.823	none
Ridge	0.516	0.951	$\alpha = 25.1$
Lasso	0.326	0.969	$\lambda = 0.0093$
Elastic net	0.328	0.969	$\lambda = 0.0103$, L1 share 0.9

Table 1: var1, all models at the chosen degree 5 (461 terms).

3.2 var2 (three inputs)

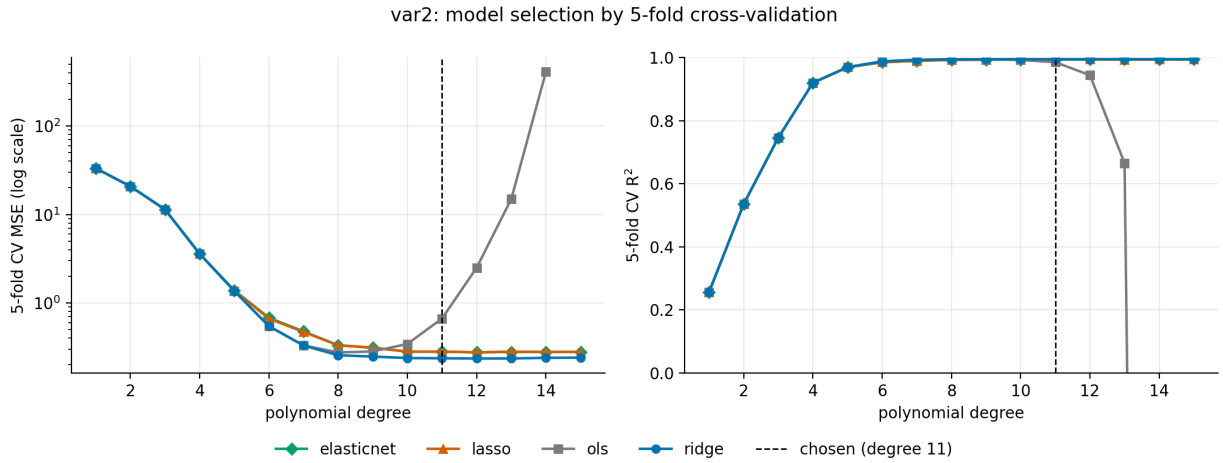


Figure 2: var2: 5-fold CV MSE (left, log scale) and CV R^2 (right) against polynomial degree. The dashed line marks the chosen degree.

Here the error keeps falling until about degree 8 and then levels off (Figure 2). The three penalised models stay flat for all higher degrees, which is what we would hope for: extra terms are shrunk instead of causing overfitting. OLS shows the opposite. It is as good as the others up to about degree 9, and then its error shoots up as the number of terms (363 at degree 11, 679 at degree 14) approaches and passes the number of rows. At degree 14 its CV MSE is above 400. This is the plot that best explains why regularisation was needed (to avoid overfitting).

The lowest CV MSE overall was at degree 12 (0.2354), but degrees 11 to 13 are practically equal. With the 1% rule the chosen model is **degree 11 with ridge**. Ridge was a little better than lasso and elastic net here, and no coefficient was set to zero. A reasonable reading is that, unlike var1, almost every term in var2 carries some signal.

Model	CV MSE	CV R^2	Penalty chosen
OLS	0.667	0.985	none
Ridge	0.237	0.995	$\alpha = 1.0$
Lasso	0.282	0.994	$\lambda = 0.0040$
Elastic net	0.281	0.994	$\lambda = 0.0044$, L1 share 0.9

Table 2: var2, all models at the chosen degree 11 (363 terms).

3.3 Summary

Problem	Degree	Model	CV MSE	CV R^2
var1	5	Lasso	0.326	0.969
var2	11	Ridge	0.237	0.995

Table 3: Final models, refitted on all 1000 training rows for the test predictions.

4 Notes and limits

- The MSE and R^2 above come from cross-validation on the training data.
- The variance of y is about 10.5 in var1 and 44.8 in var2. A CV MSE of 0.33 and 0.24 means most of the variance is explained, but there is still some error left that the model does not capture. In var2 the error stops improving at around 0.24 for many degrees in a row, so it is probably close to the noise level in the data.
- In var2 the search stopped at degree 15, not 20, because the error had not improved for three degrees. Degrees 16 to 20 were therefore not tested. The same applies to var1, which was tested up to degree 8 and not 10. Given the flat or rising curves I do not expect a better result there, hence it was not checked (since MSE is expected to form a U shaped curve of sorts).
- Standardisation uses the mean and standard deviation of all training rows, and not of each fold separately. Only the inputs are involved, never y , so the effect is tiny.

5 Code

All code is in the GitHub repository. A single script, `run.py`, runs the whole procedure for both problems: it builds the features, runs the cross-validation, draws the two figures, writes the CV tables, and saves the prediction files `IMT2024014_pred_var1.csv` and `IMT2024014_pred_var2.csv`. It uses numpy, pandas, scikit-learn and matplotlib.